

Node allocator benchmark

Page allocators vs. the former default pool, for `boost::root_ptr`, `std::unique_ptr` and `std::shared_ptr`

Measured 2026-09-27 on an Intel Core i7-4700HQ (4 cores / 8 threads), Linux 5.15, clang 23 -02, libstdc++ 12, glibc 2.35. Median of 5 runs; every combination runs in a fresh process. Five builds: with thread support (default), `BOOST_DISABLE_THREADS`, `BOOST_ROOT_PTR_STRIPED_LOCKS`, `BOOST_PAGE_ALLOCATOR_THREAD_CACHE`, and the last two together. Full tables and method: `ALLOCATOR_BENCHMARK.md`.

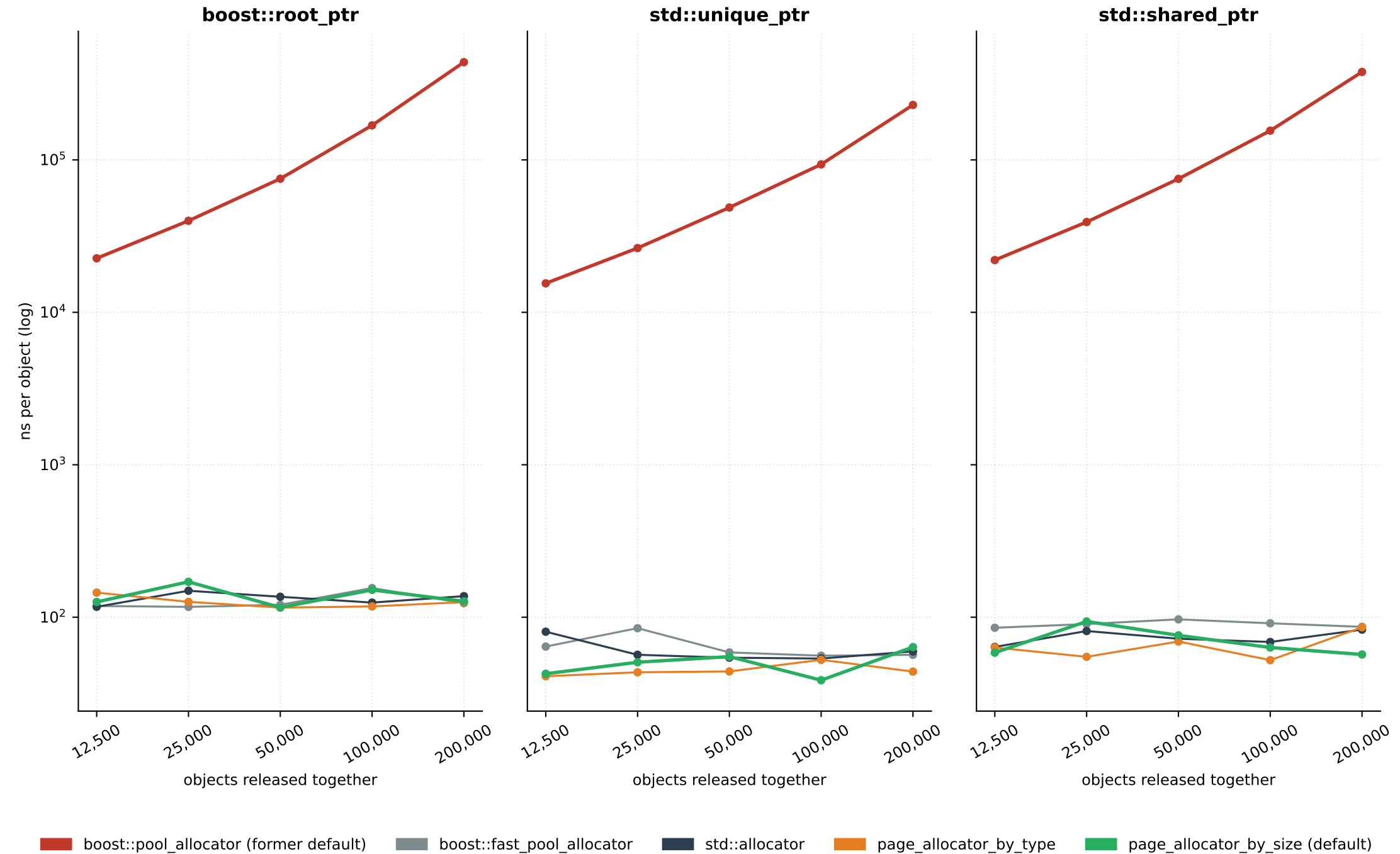
Findings

- `boost::pool_allocator`, the former default, is quadratic on release (`ordered_free`): releasing 200,000 objects costs 229-438 us per object; the others stay at 39-171 ns.
- The page allocators are fastest on bulk, mixed and cycle release; `page_allocator_by_size`, now the default, uses 9-19% less memory than `boost::pool_allocator`.
- Per-thread caches (opt-in): 4 threads go 157 -> 1.9 ns per `unique_ptr` and 149 -> 2.4 ns per `shared_ptr` (`std::allocator`: 4.1 / 4.8); striped `root_ptr` 289 -> 27 ns; churn 2.3-3.1x.
- Thread support is `root_ptr`'s largest cost: without it, churn drops 56.2 -> 9.4 ns (6.0x), cycles 120 -> 46 ns per node.
- Striped locks: with 4 threads and `std::allocator`, `root_ptr` goes 271 -> 30 ns (9.0x), but single-threaded code is 1.33-3.35x slower. Off by default.
- `page_allocator_by_type` costs 2.4-4.9x the time and 1.6-1.8 MB more memory than `by_size` with many sparse types: each type fills a 64 KiB page of its own.
- `shared_node_ptr` (`FCXXSS_SHARED_PTR`, measured 2026-09-28 against `root_ptr`): churn equal, release up to 1.2x slower, 16-25% more memory; 4 threads 1.8x faster (24x with thread caches).

Pages (with thread support unless noted)

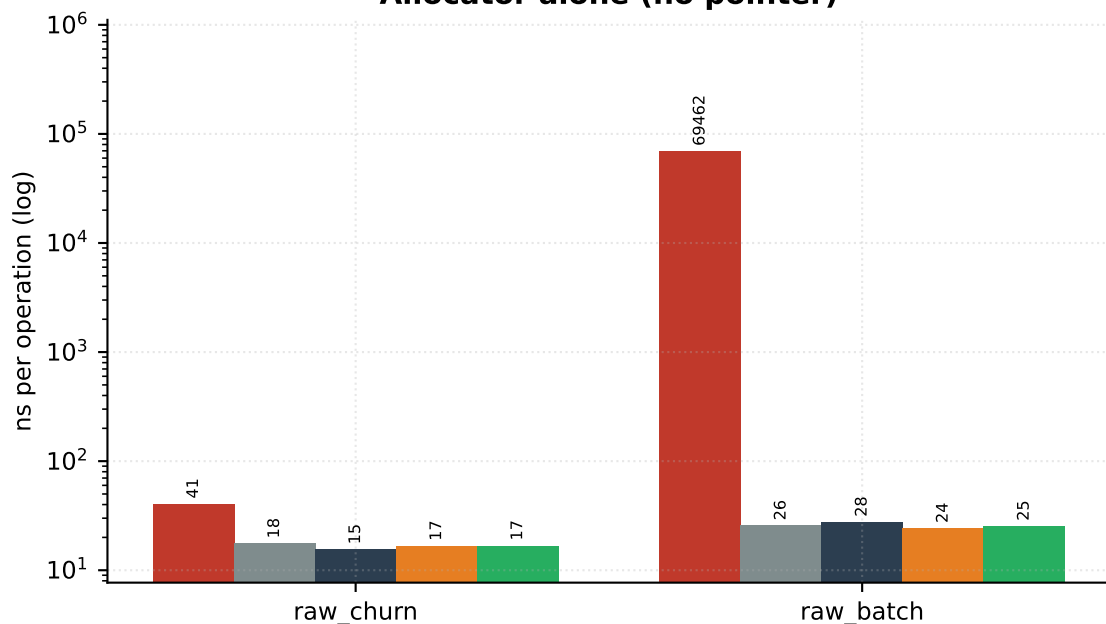
2 Bulk release scaling	3 Time per scenario	4 Resident memory
5 Pointer types compared	6 Threads	7 Without thread support
8 What thread support costs	9 Striped locks (<code>root_ptr</code>)	10 Per-thread caches
11 Four threads, every build	12 <code>shared_node_ptr</code> against <code>root_ptr</code>	

Bulk release scaling: boost::pool_allocator grows with the number of objects, the others stay flat (lower is better)

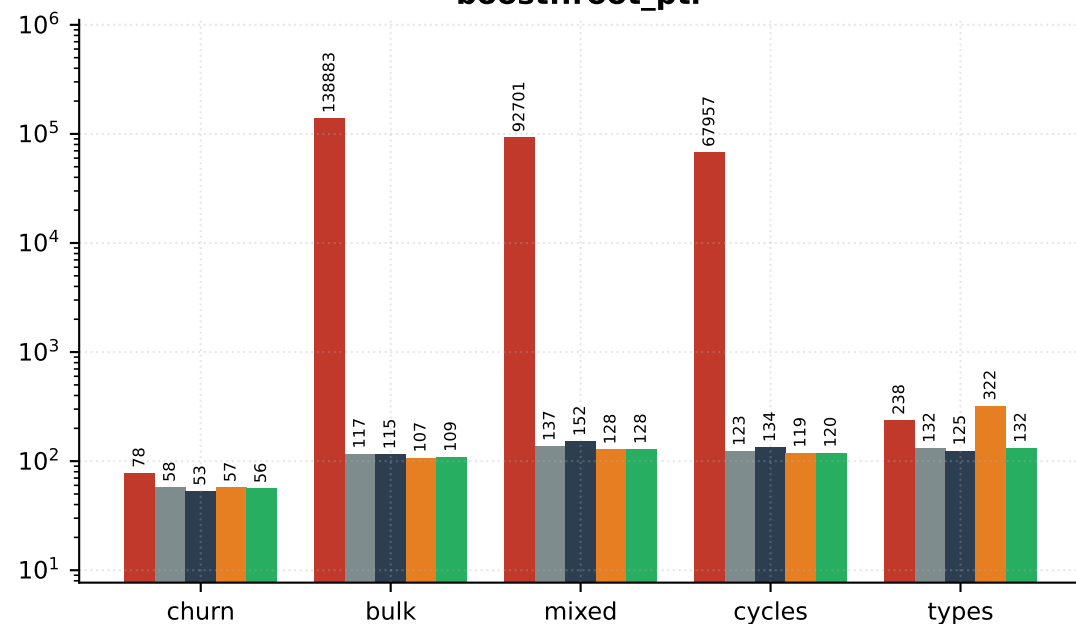


Time per operation, single-threaded scenarios, built with thread support (lower is better)

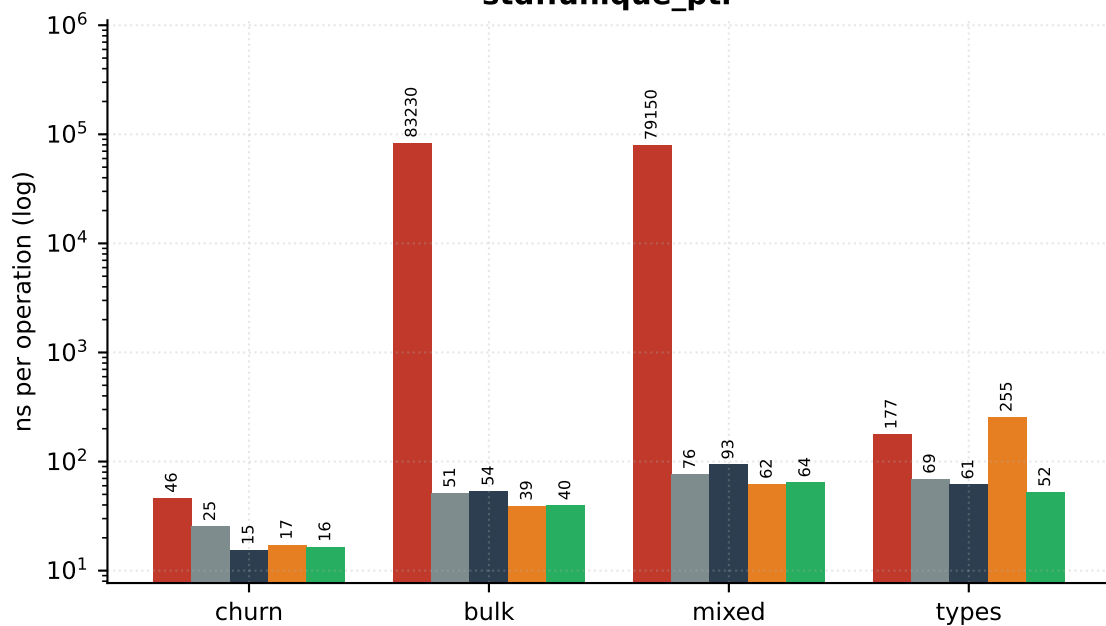
Allocator alone (no pointer)



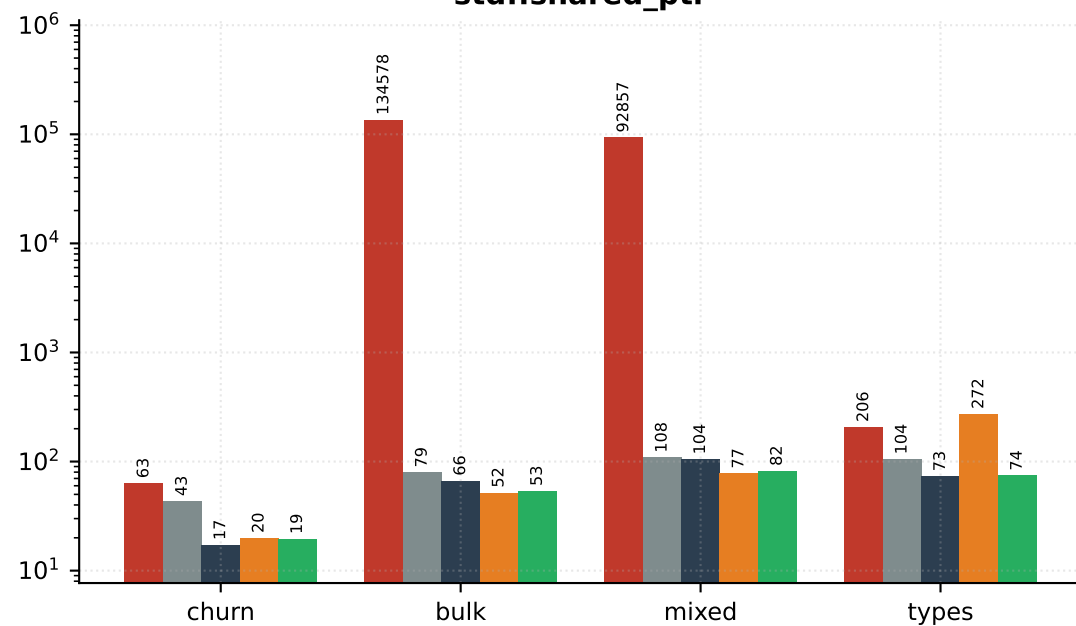
boost::root_ptr



std::unique_ptr



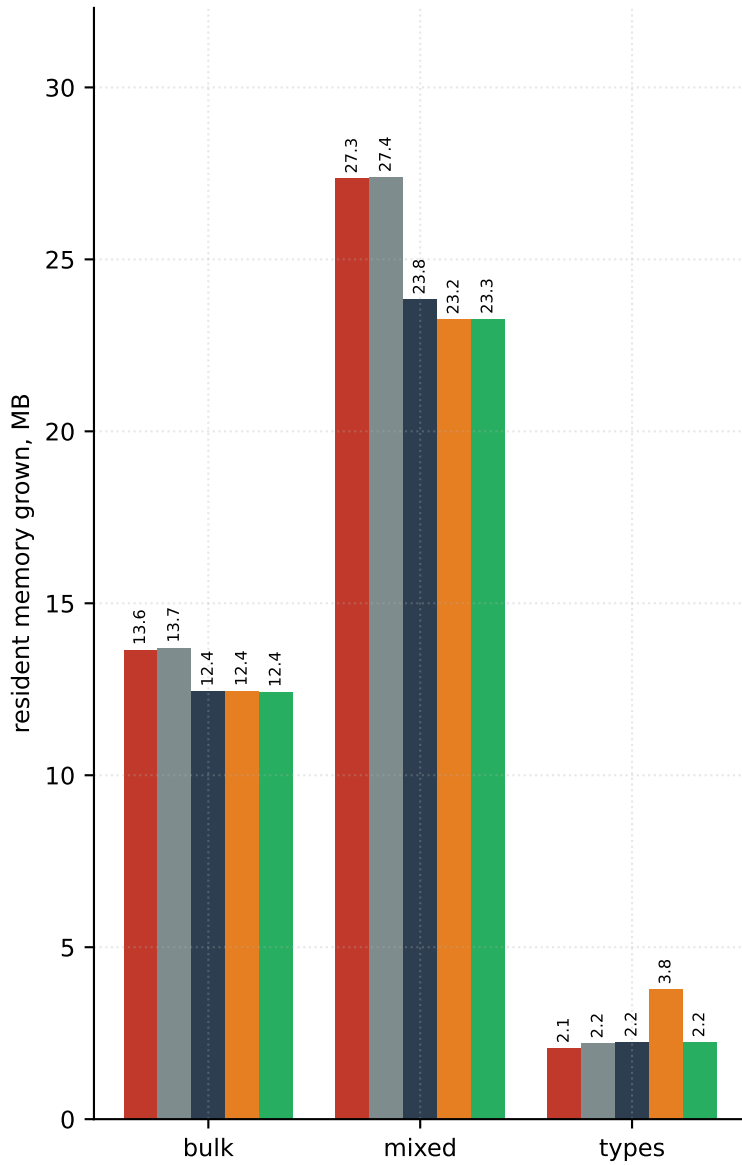
std::shared_ptr



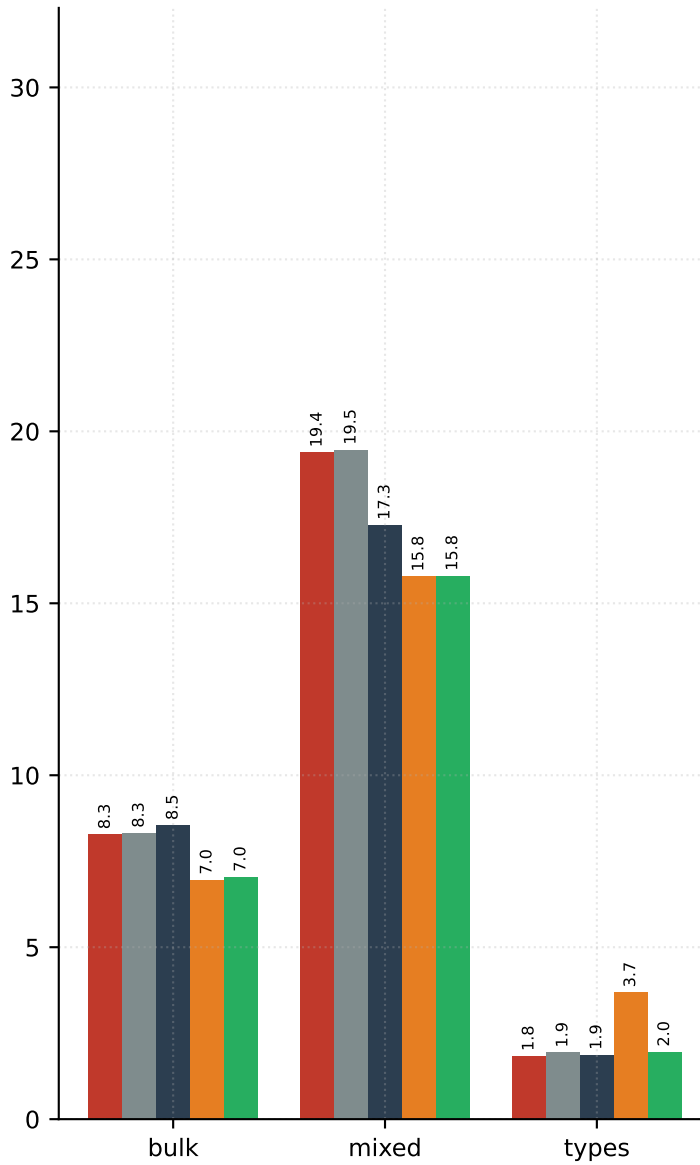
boost::pool_allocator (former default) boost::fast_pool_allocator std::allocator page_allocator_by_type page_allocator_by_size (default)

Resident memory while the objects are live (lower is better)
bulk: 100,000 x 48 B mixed: 50,000 each of 16, 48 and 200 B types: 32 types x 100 objects

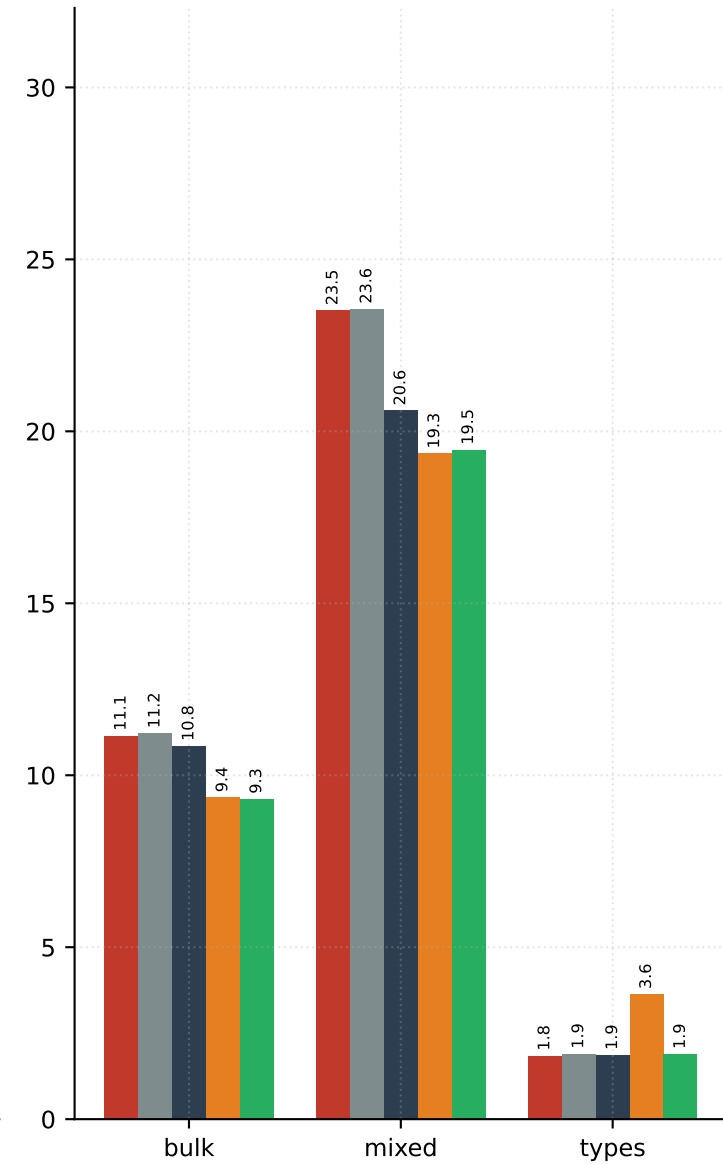
boost::root_ptr



std::unique_ptr



std::shared_ptr

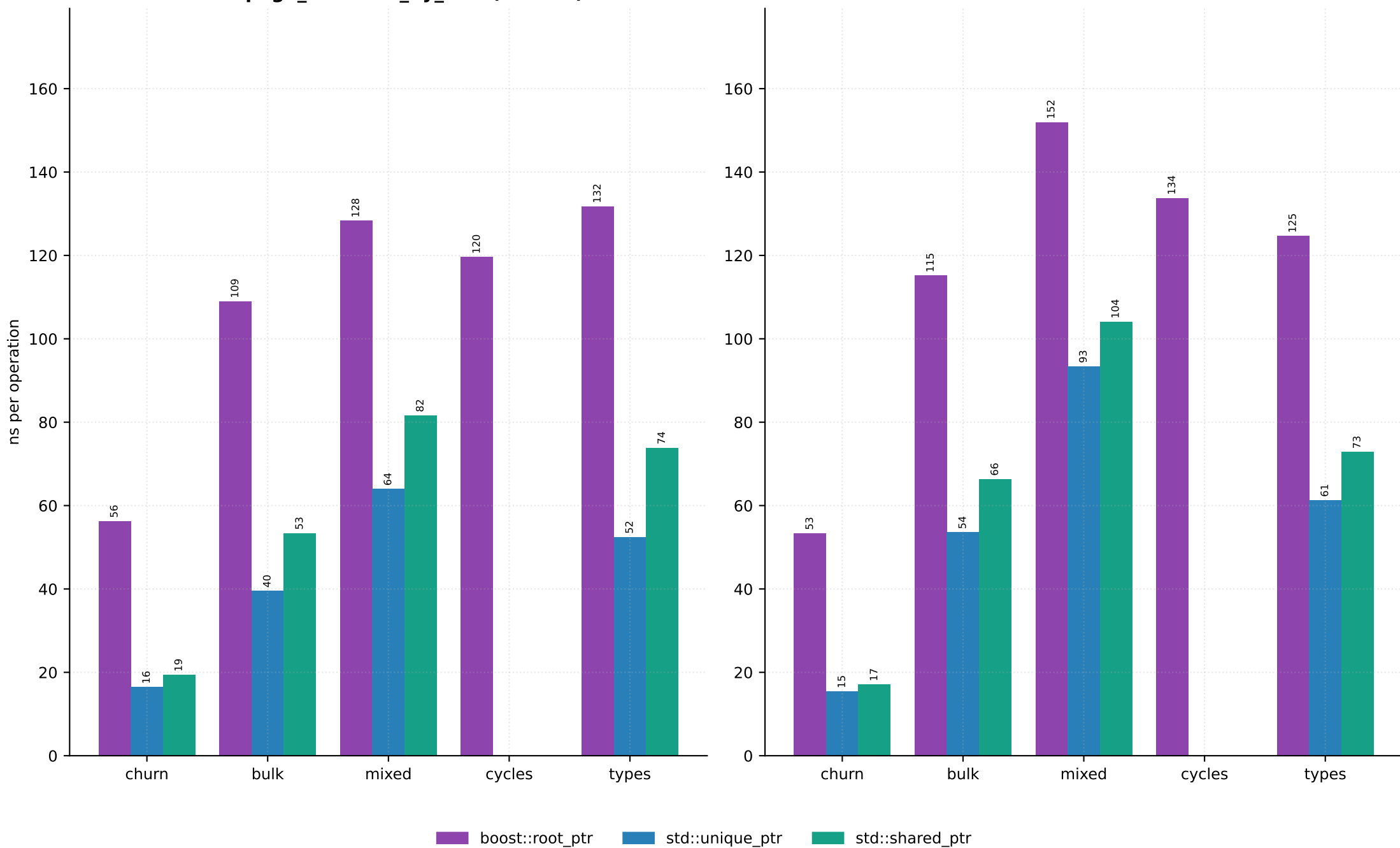


boost::pool_allocator (former default) boost::fast_pool_allocator std::allocator page_allocator_by_type page_allocator_by_size (default)

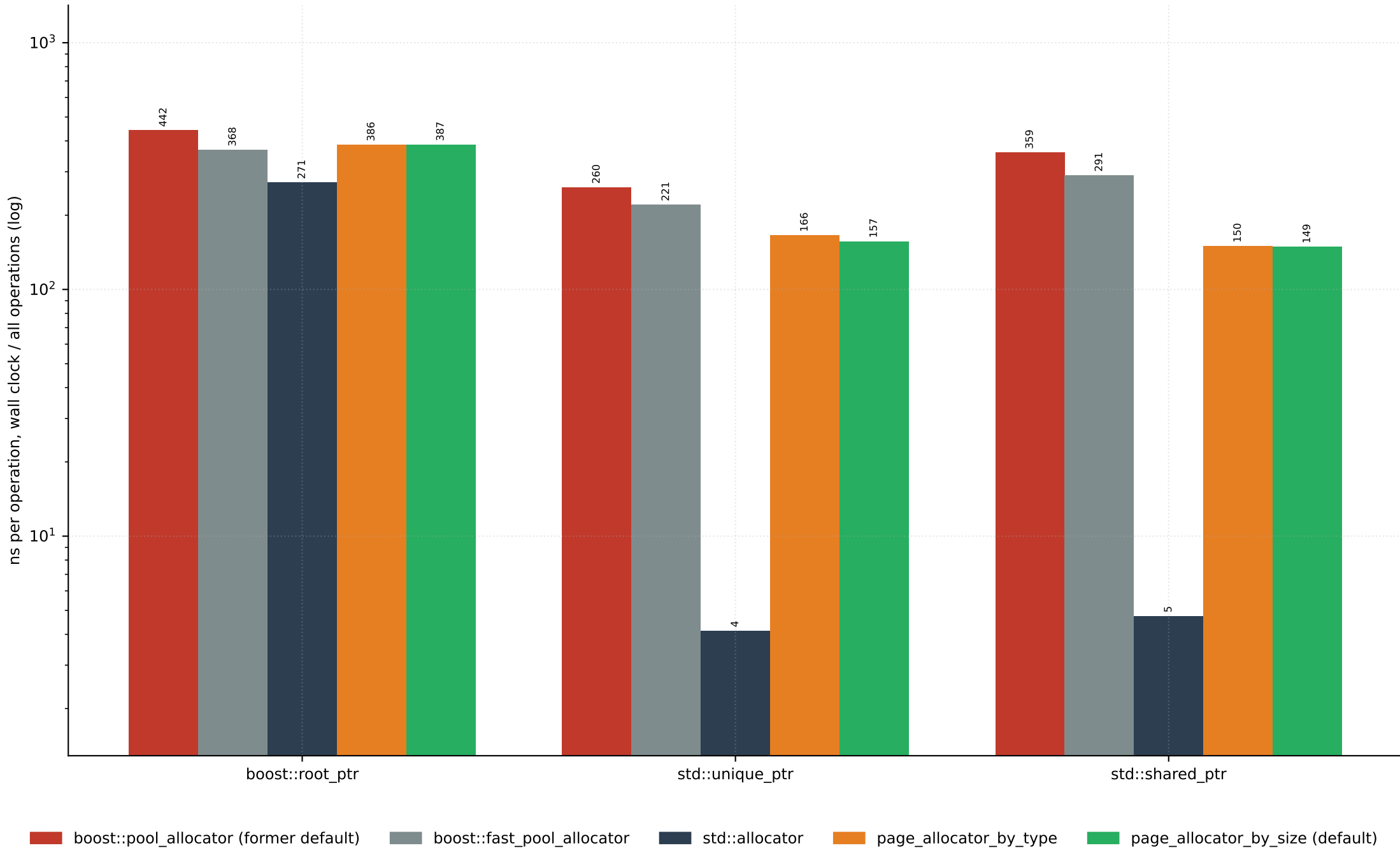
Pointer types compared, time per operation (lower is better)
cycles: root_ptr only - a unique_ptr cannot form one, a shared_ptr cycle is never freed

with page_allocator_by_size (default)

with std::allocator

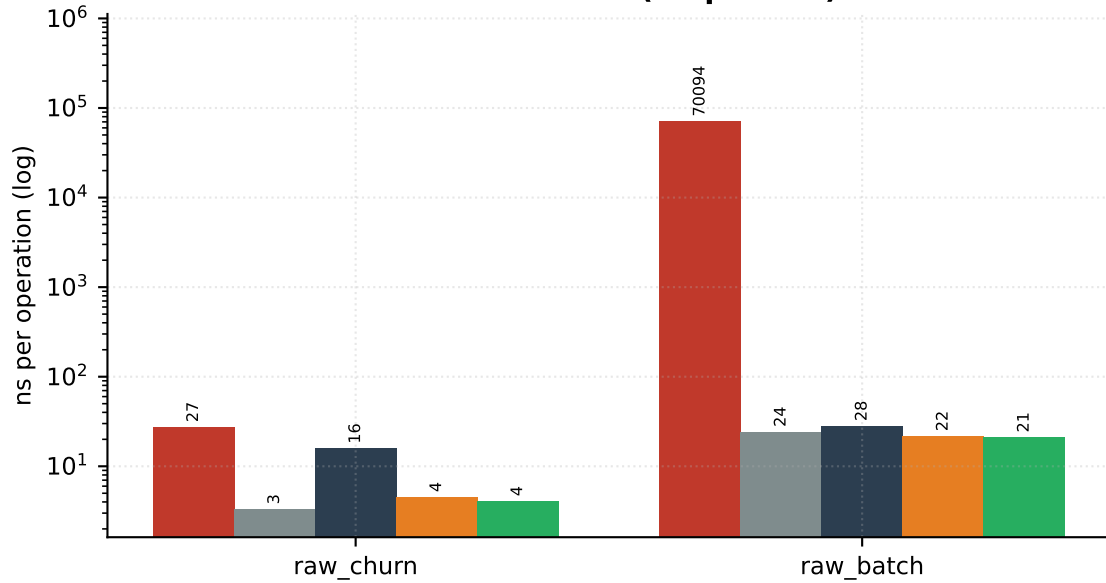


4 threads, each creating and dropping 250,000 objects (lower is better)
*without BOOST_PAGE_ALLOCATOR_THREAD_CACHE: glibc's per-thread caches take no lock; each page pool has one mutex;
every root_ptr operation takes a global mutex*

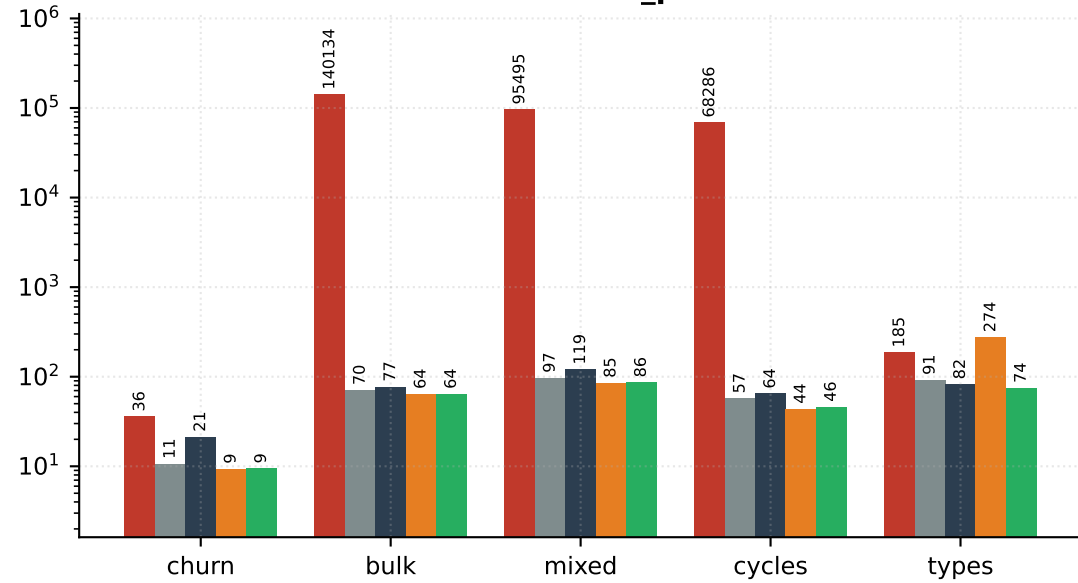


**Time per operation, built with BOOST_DISABLE_THREADS (lower is better)
no root_ptr mutex, plain reference counts, no pool mutexes**

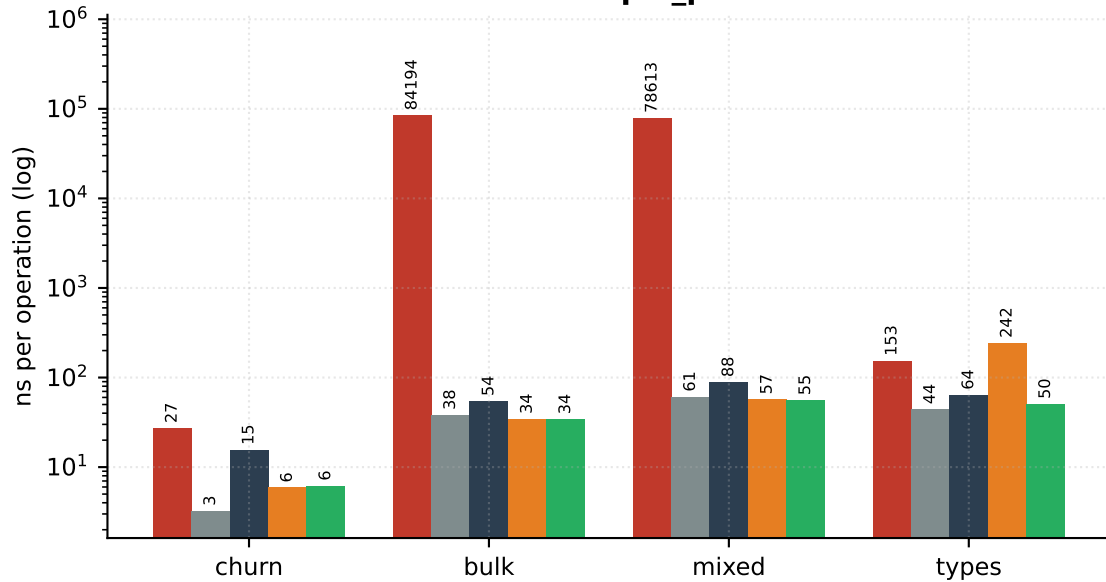
Allocator alone (no pointer)



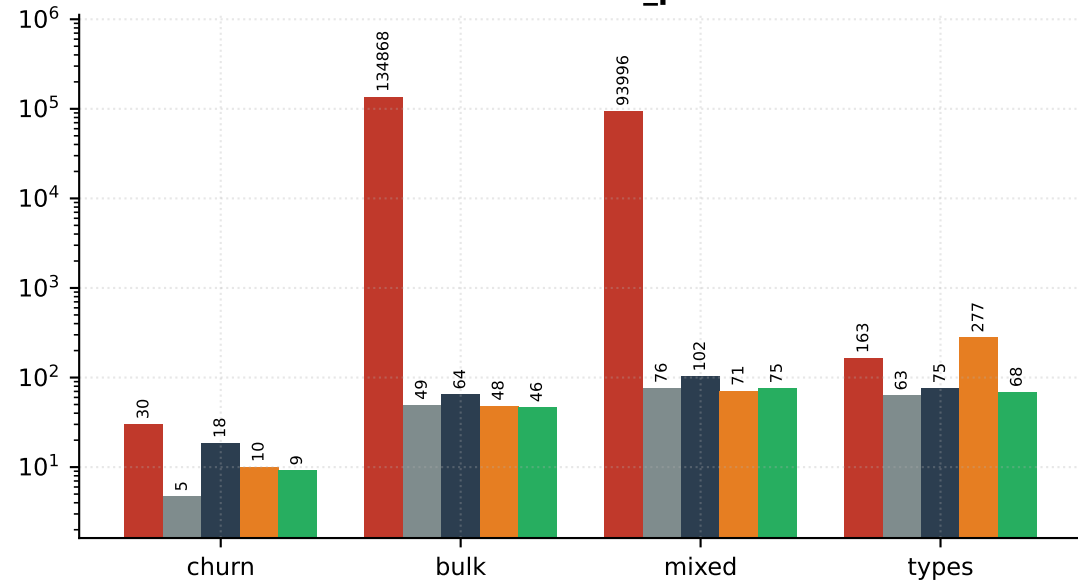
boost::root_ptr



std::unique_ptr



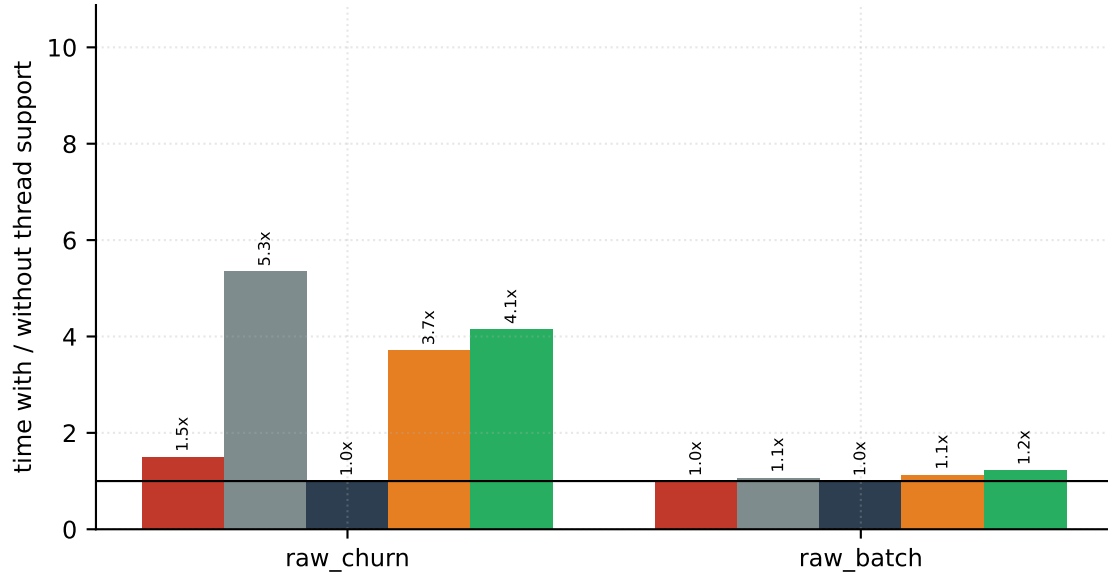
std::shared_ptr



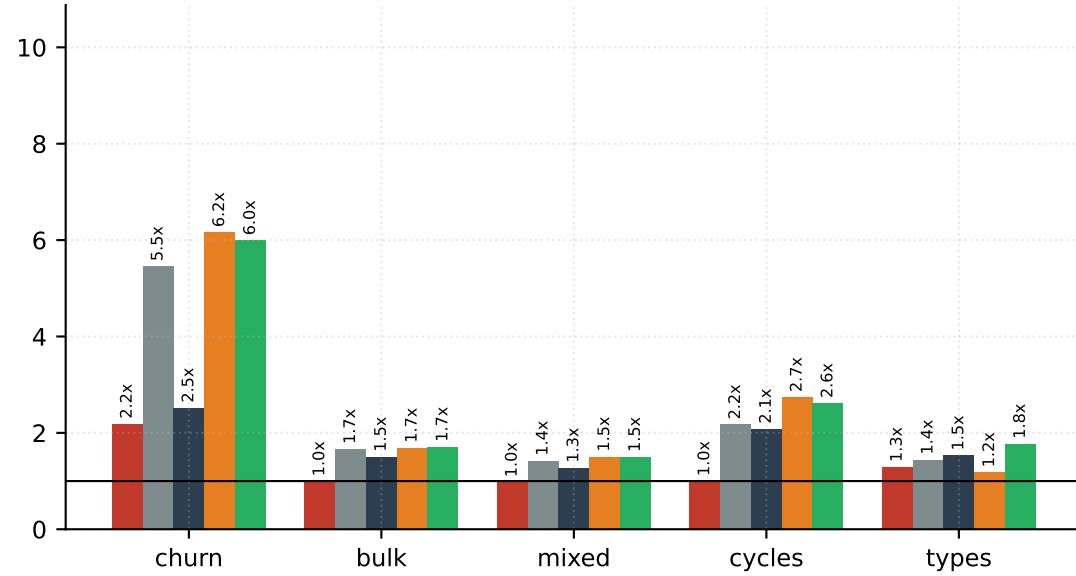
boost::pool_allocator (former default) boost::fast_pool_allocator std::allocator page_allocator_by_type page_allocator_by_size (default)

What thread support costs: time with thread support divided by time with BOOST_DISABLE_THREADS (lower is better)
1.0x = thread support costs nothing; above 1.0x it makes the code slower

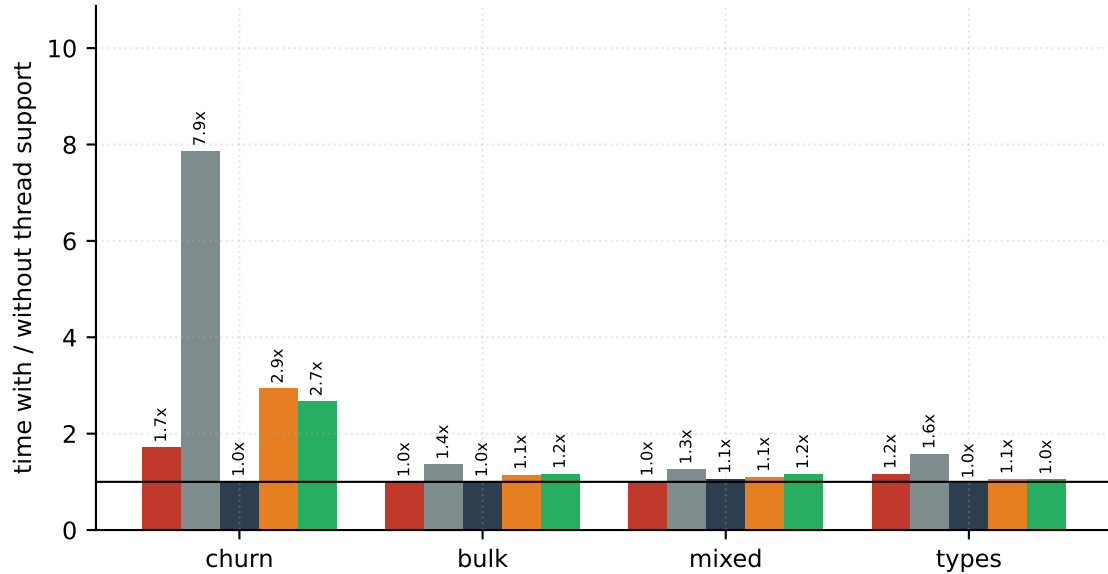
Allocator alone (no pointer)



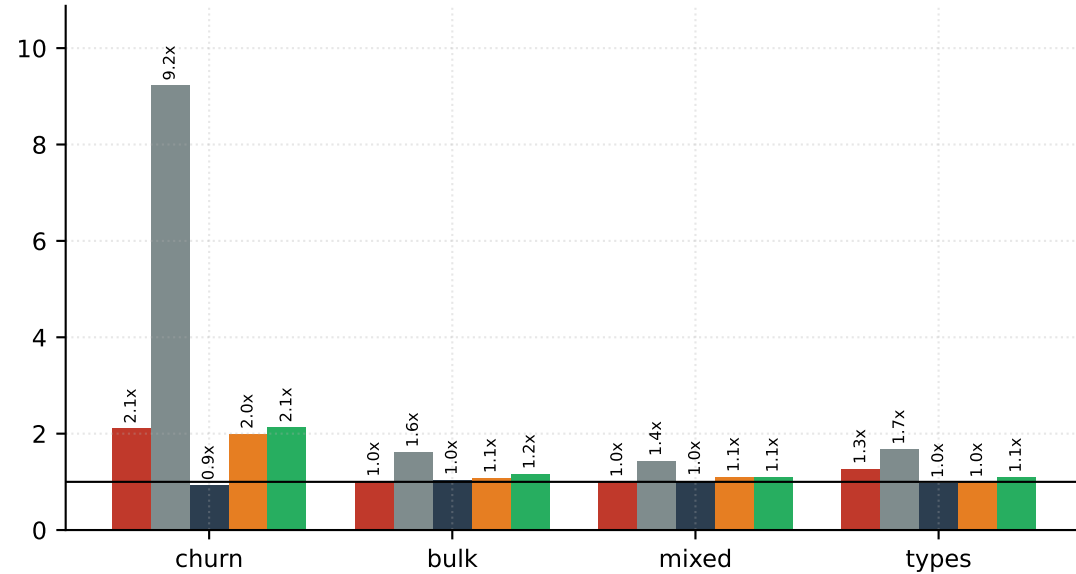
boost::root_ptr



std::unique_ptr



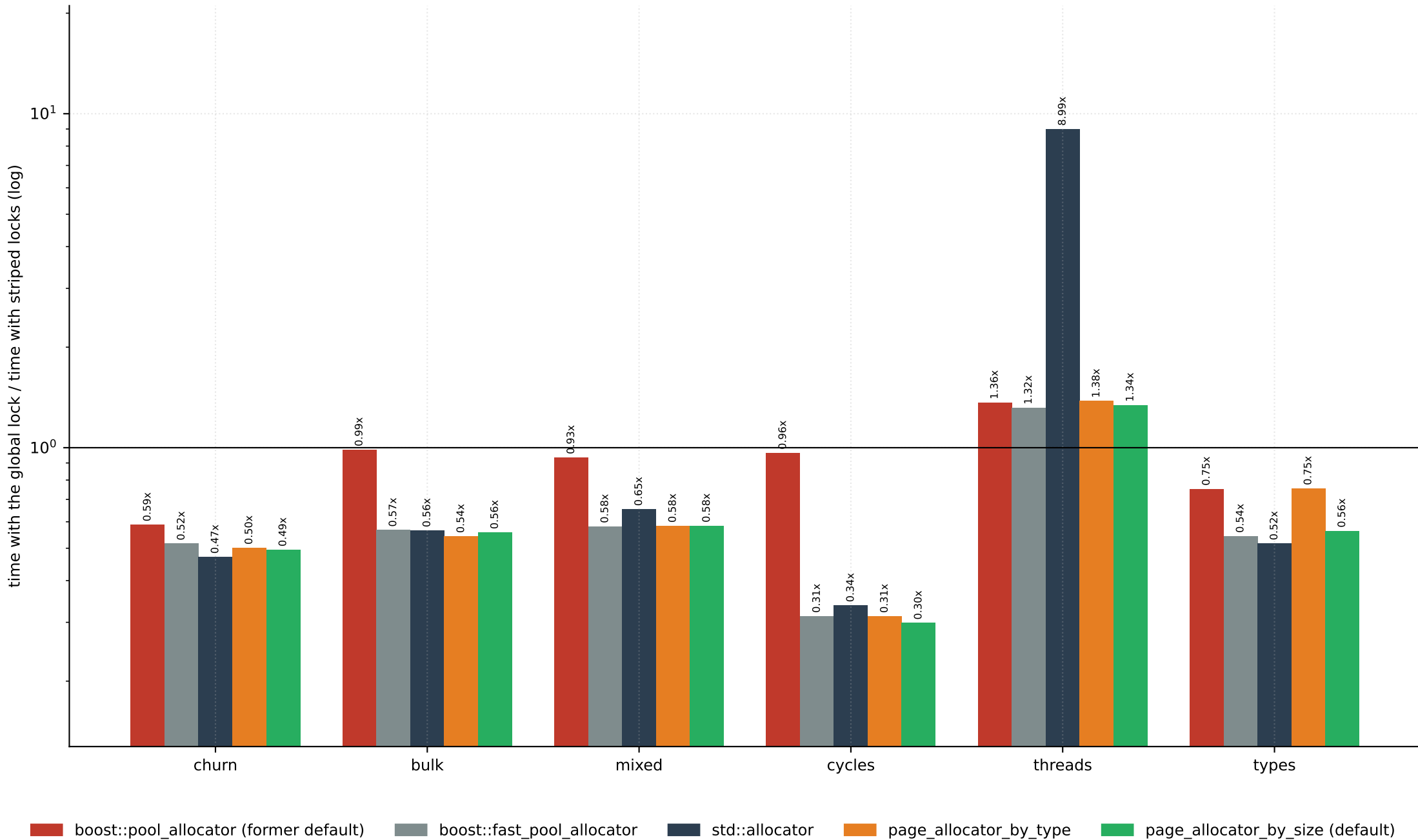
std::shared_ptr



boost::pool_allocator (former default) boost::fast_pool_allocator std::allocator page_allocator_by_type page_allocator_by_size (default)

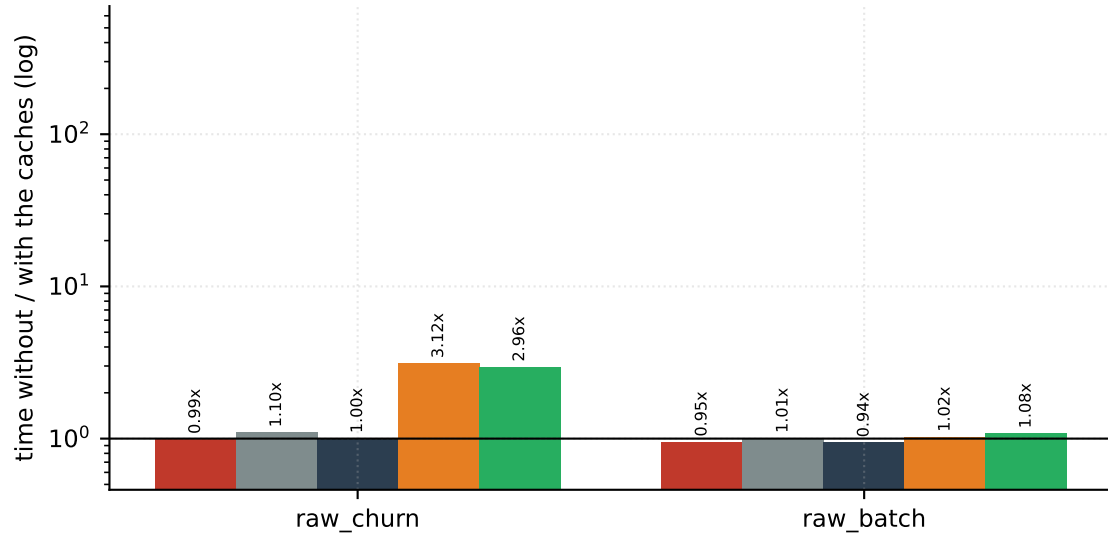
boost::root_ptr with BOOST_ROOT_PTR_STRIPED_LOCKS (higher is better)
time with the global lock / time with striped locks: above 1.0x the striped locks are faster

about 8 lock operations per object instead of 3: slower single-threaded, faster with four threads unless the allocator itself serializes

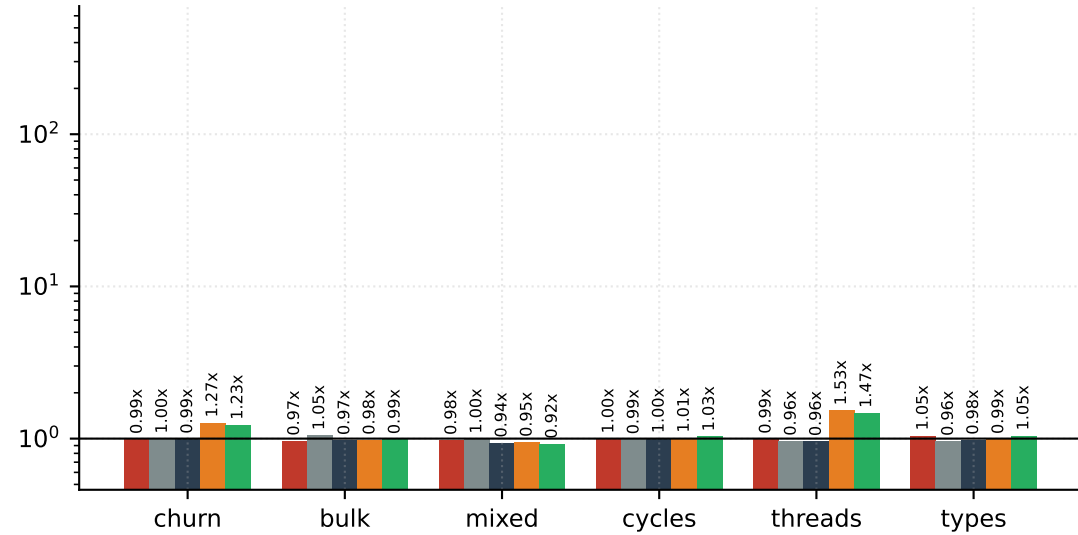


Per-thread caches (BOOST_PAGE_ALLOCATOR_THREAD_CACHE) (higher is better)
time without / with the caches: above 1.0x the caches are faster
only the page allocators have them; the other allocators are the control

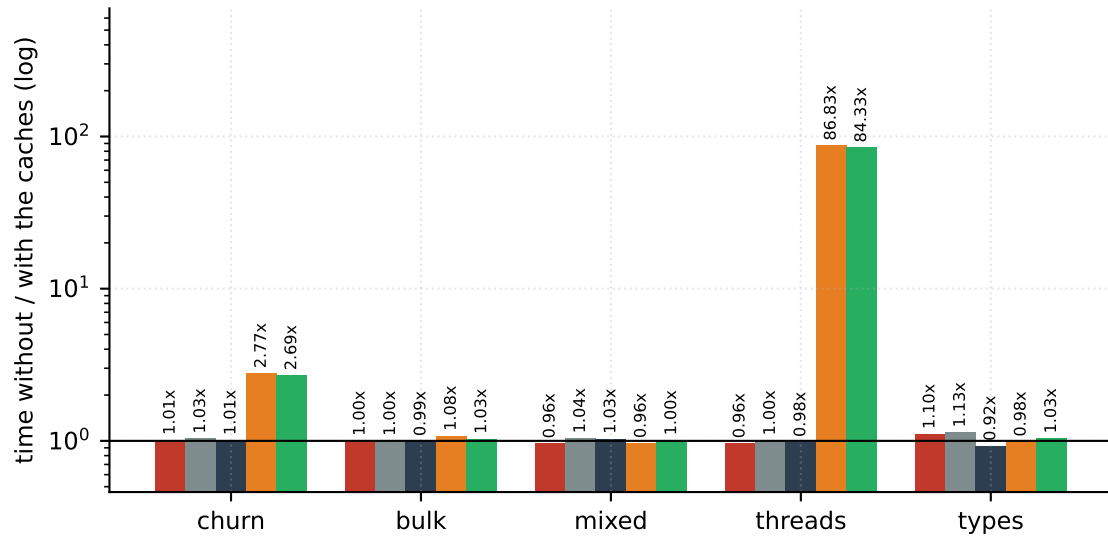
Allocator alone (no pointer)



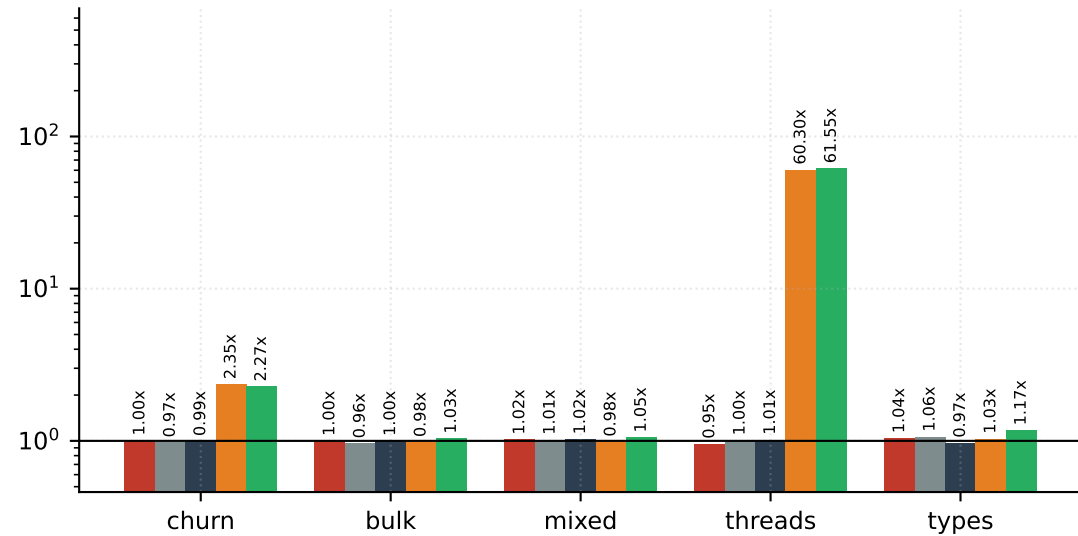
boost::root_ptr



std::unique_ptr



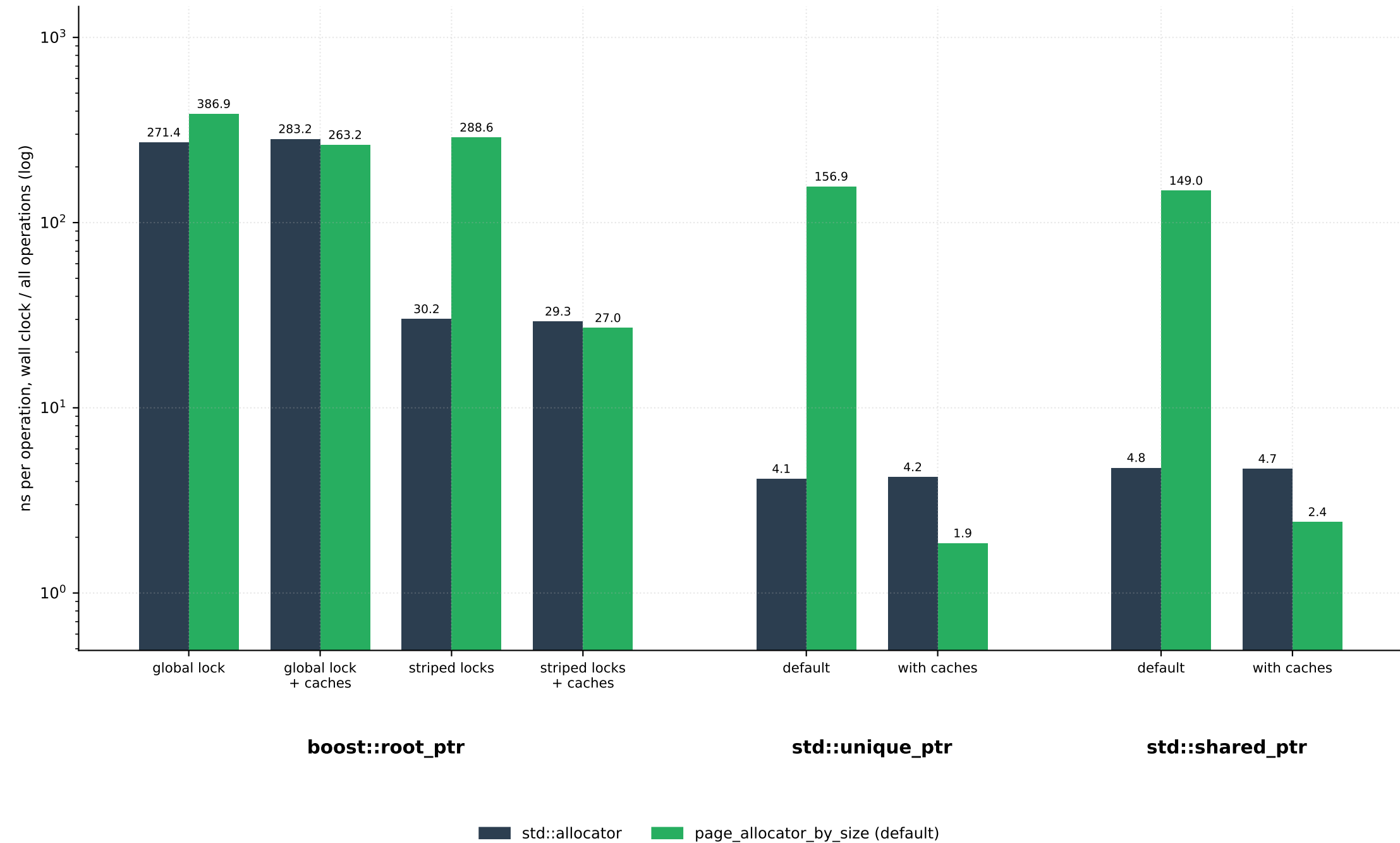
std::shared_ptr



■ boost::pool_allocator (former default)
 ■ boost::fast_pool_allocator
 ■ std::allocator
 ■ page_allocator_by_type
 ■ page_allocator_by_size (default)

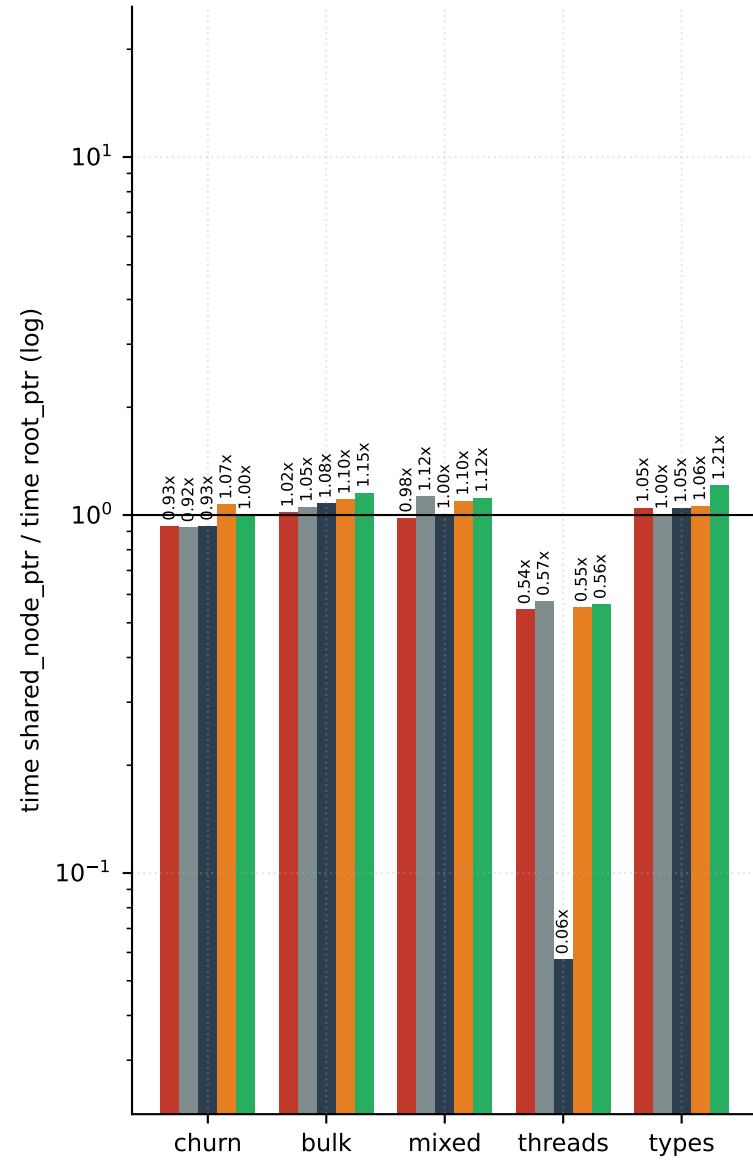
4 threads, each creating and dropping 250,000 objects, in every build (lower is better)

striped locks apply to root_ptr only; the caches are the page allocators' (std::allocator is the control)

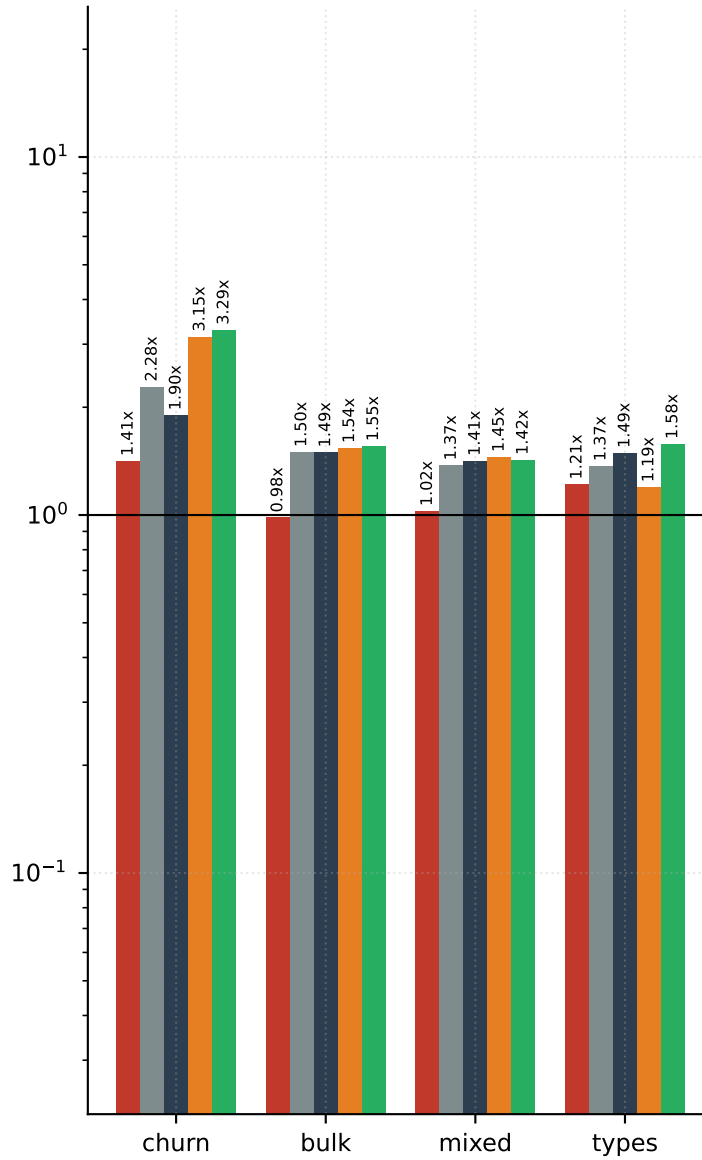


boost::shared_node_ptr (FCXXSS_SHARED_PTR) against boost::root_ptr (lower is better)
below 1.0x shared_node_ptr is faster; one run of their own - cycles: a shared_ptr never frees one

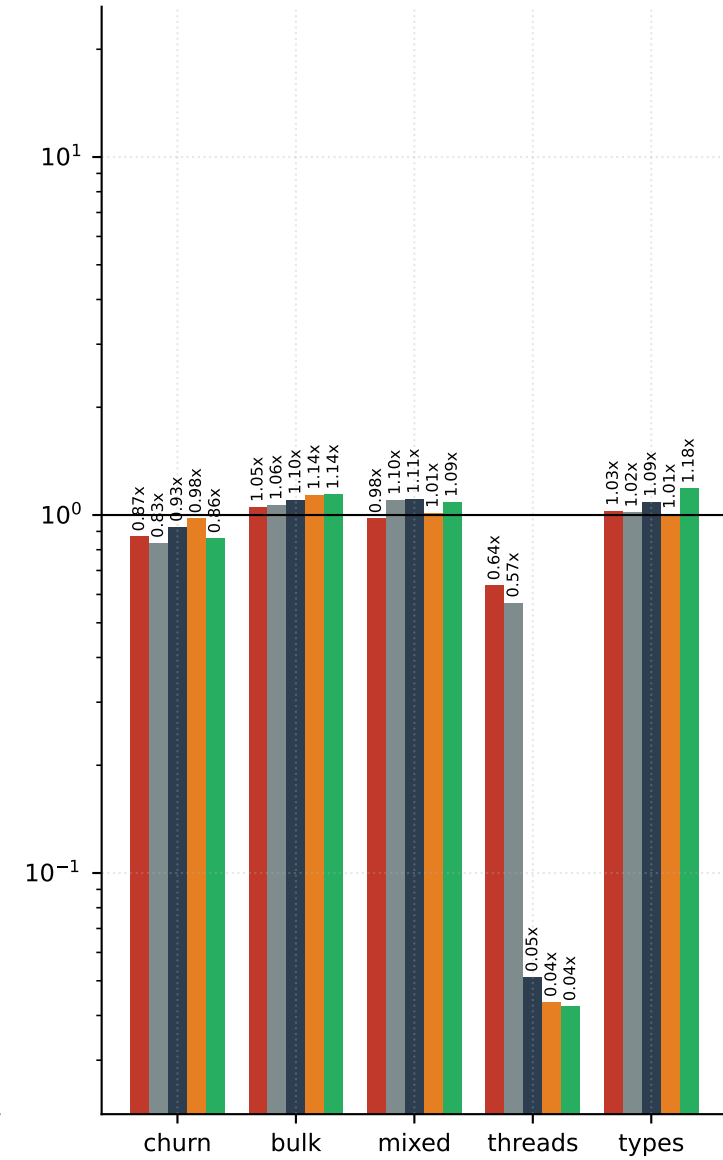
with thread support



BOOST_DISABLE_THREADS



thread caches



boost::pool_allocator (former default) boost::fast_pool_allocator std::allocator page_allocator_by_type page_allocator_by_size (default)